

Compiler Design Labsheet 3

Name: G Tejaswini

Roll No.: AM.SC.U4CSE23219

Sample Program:

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (expr (expr (term a)) + (term b)) + (term c)) <EOF>)
```

Exercises

1. Modify the grammar to accept the inputs:

i. a-b-c

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (expr (expr (term a)) - (term b)) - (term c)) <EOF>)
```

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (expr (expr (term a)) + (term b)) - (term c)) <EOF>)
```

iii. a*b

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (expr (term a)) * (term b)) <EOF>)
```

iv. x+y*z:(Ensure that * has more precedence than +)

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (term (factor x)) + (expr (term (factor y) * (factor z)))) <EOF>)
```

v. a/b

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (term (factor a) / (factor b))) <EOF>)
```

a+b/c

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr  
(prog (expr (term (factor a)) + (term (factor b) / (factor c))) <EOF>)
```

vii. (a+b)

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 ( (expr (term (factor (factor1 a))) + (term (factor (factor1 b))))
)))) <EOF>)
```

(a+b)*c

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 ( (expr (term (factor (factor1 a))) + (term (factor (factor1 b))))
))) * (factor (factor1 c)))) <EOF>)
```

a*(b+c)

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 a)) * (factor (factor1 ( (expr (term (factor (factor1 b))) + (term
(factor (factor1 c)))) )))) <EOF>)
```

((a+b)*c)

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 ( (expr (term (factor (factor1 ( (expr (term (factor (factor1 a)))
+ (term (factor (factor1 b)))) )))) * (factor (factor1 c)))) )))) <EOF>)
```

2. Modify the given grammar to support identifiers and numeric literals.

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 2))) + (term (factor (factor1 2.23)))) <EOF>)
```

3. Extend the grammar to recognize floating point numbers of the form: digits '.' digits

```
grammar Expr;
prog : expr EOF ;
expr : term (('+' | '-') term)* ;
term : factor (('*' | '/') factor)* ;
factor : factor1 ('^' factor1)* ;
factor1 : ID | NUMBER | '(' expr ')' ;
ID      : [a-zA-Z]+ ;
NUMBER  : [0-9]+ '.' [0-9]+ | [0-9]+ ;
WS      : [ \t\r\n]+ -> skip ;
```

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 22.53)))) <EOF>)
```

4. Modify grammar so operands can be: identifier | integer | float

Identifier:

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 x)))) <EOF>)
```

Int

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 6)))) <EOF>)
```

Float:

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (expr (term (factor (factor1 6.66)))) <EOF>)
```

5. Write grammar for the following: i. $x = 5$ ii. $y = a + b$

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (stmt x = (expr (term (factor 5))) ;) <EOF>)
```

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (stmt y = (expr (term (factor a)) + (term (factor b)))) <EOF>)
```

6. Modify the assignment grammar so that it accepts multiple statements $x=10$ $y=20$ $z=x+y$

```
C:\Users\exam\23219\labsheet3>java -cp .;antlr-4.13.2-complete.jar TestExpr
(prog (stmt x = (expr (term (factor 10)))) (stmt y = (expr (term (factor 20))))
(stmt z = (expr (term (factor x)) + (term (factor y)))) <EOF>)
```